

# Capítulo 3: Aprendizaje No Supervisado

Universidad de Salamanca

Facultad de Ciencias

Programación Avanzada

Diego M. Jiménez-Bravo · [dmjimenez@usal.es](mailto:dmjimenez@usal.es)

- 1 Introducción
- 2 3.1 Clustering
- 3 3.2 Reducción de dimensionalidad
- 4 3.3 Detección de anomalías

## 1 Introducción

## 2 3.1 Clustering

## 3 3.2 Reducción de dimensionalidad

## 4 3.3 Detección de anomalías

# ¿Qué es el aprendizaje no supervisado?

A diferencia de los capítulos anteriores, aquí el entrenamiento **no está guiado** por ninguna etiqueta o variable objetivo. El sistema no recibe un criterio de “verdad fundamental” (*ground-truth*); en su lugar, debe explorar las relaciones matemáticas intrínsecas de los datos para revelar su estructura oculta. Técnicamente, mientras el aprendizaje supervisado busca aproximar la relación  $X \rightarrow y$ , el no supervisado trabaja “a ciegas” para descubrir la **estructura latente** de los propios datos  $X$ , sin ninguna respuesta correcta de referencia.

Como ilustró el científico Yann LeCun: *“Si la inteligencia fuera un pastel, el aprendizaje no supervisado sería el bizcocho, el supervisado sería la cobertura, y el aprendizaje por refuerzo sería la cereza”*. La inmensa mayoría de los datos del mundo real no vienen etiquetados, lo que hace de este paradigma un pilar fundamental del aprendizaje automático.

Existen tres grandes familias de técnicas: **clustering** (agrupar datos similares), **reducción de dimensionalidad** (simplificar datos complejos) y **detección de anomalías** (identificar casos raros).

- **Segmentación de mercado:** agrupar clientes en nichos homogéneos según sus datos demográficos y de consumo, sin definir las categorías de antemano, para dirigir campañas específicas.
- **Visualización de datos complejos:** proyectar conjuntos de datos con cientos de variables a representaciones en 2D o 3D, preservando su estructura, para que una persona pueda entenderlos de un vistazo.
- **Sistemas de recomendación:** agrupar usuarios con gustos afines o productos con características similares para predecir preferencias implícitas.
- **Detección de anomalías:** modelar el comportamiento “normal” de los datos para identificar automáticamente fraudes financieros o fallos en líneas de fabricación.

**Ejemplo:** una tienda online que agrupa a sus clientes por hábitos de compra (sin decirle previamente qué grupos buscar) para después diseñar ofertas distintas para cada grupo — eso es *clustering* aplicado a segmentación de mercado.

1 Introducción

2 3.1 Clustering

3 3.2 Reducción de dimensionalidad

4 3.3 Detección de anomalías

# ¿Qué es el *clustering*?

El *clustering* (o agrupamiento) es la técnica más extendida del aprendizaje no supervisado. Su objetivo es **particionar** un conjunto de datos en subgrupos cuyos miembros compartan una **alta similitud interna** y, al mismo tiempo, presenten una **alta diferenciación** respecto a los miembros de otros subgrupos.

Existen distintas familias de algoritmos, según cómo definan la pertenencia a un grupo: basados en **centroides** (*K-means*), basados en **jerarquías** (*clustering* jerárquico) y basados en **densidad** (DBSCAN). Los estudiaremos uno por uno.

# *K-means*: mecánica del algoritmo

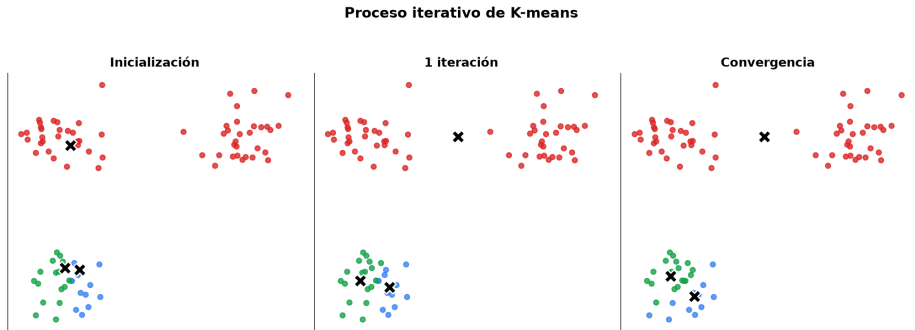
El *K-means* es un algoritmo de agrupación basado en **centroides**: busca particionar un conjunto de observaciones en  $K$  *clusters* distintos, donde cada centroide actúa como el “centro de gravedad” de su grupo. Su meta matemática es minimizar la **inercia intracluster** (WCSS, *Within-cluster Sum of Squares*). El proceso se repite iterativamente:

- ➊ **Inicialización**: se eligen  $K$  puntos aleatorios como centroides iniciales.
- ➋ **Asignación**: cada muestra se une al centroide más cercano (normalmente, por distancia euclídea).
- ➌ **Actualización**: cada centroide se recalcula como la media de todas las muestras asignadas a su grupo.
- ➍ Se repiten los pasos 2 y 3 hasta que los centroides dejan de moverse: la **convergencia**.



# K-means: proceso visual

La siguiente figura muestra las tres etapas sobre un conjunto de datos con tres grupos naturales: los centroides (marcados con X) parten de posiciones aleatorias y convergen hacia el centro de cada grupo tras pocas iteraciones.



**Ejemplo:** es como asignar mesas en una fiesta: cada invitado se sienta en la mesa más cercana; luego cada mesa se recoloca al centro de sus invitados; y se repite el proceso hasta que nadie tiene que

## *K-means++*: el problema de la inicialización

El *K-means* es un algoritmo **heurístico** y, por tanto, muy sensible a la inicialización aleatoria de sus centroides. Si los centroides iniciales caen en regiones desfavorables, el algoritmo puede converger en un **mínimo local subóptimo** — un agrupamiento que parece bueno localmente pero no es el mejor posible.

Para mitigar este problema se usa la inicialización *K-means++*:

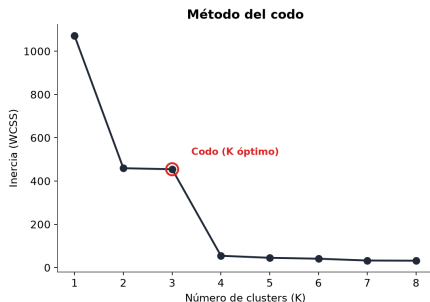
- El primer centroide se elige al azar entre los puntos de datos.
- Cada centroide siguiente se elige de forma probabilística, dando más probabilidad a los puntos que están **más lejos** de los centroides ya elegidos.

Este “sembrado inteligente” asegura que los centroides iniciales queden bien repartidos por el espacio, acelerando la convergencia y mejorando la calidad del resultado final.

# El método del codo

Como el número de *clusters*  $K$  es un hiperparámetro que hay que fijar antes de entrenar, necesitamos un criterio para elegirlo. El **método del codo** consiste en graficar la inercia (WCSS) frente a distintos valores de  $K$ :

- A medida que  $K$  aumenta, la inercia disminuye de forma natural (los grupos son más pequeños y compactos).
- El valor óptimo de  $K$  está en el punto donde la curva deja de caer bruscamente y se vuelve casi plana: el “codo”.



# Clustering jerárquico: enfoque aglomerativo

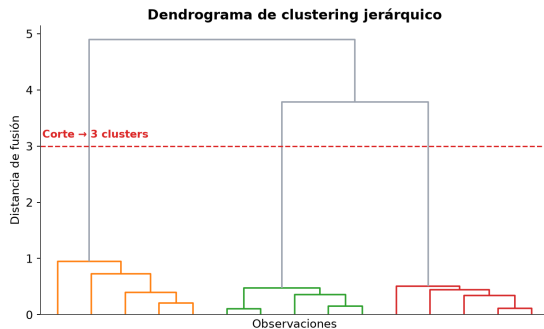
El *clustering* jerárquico agrupa datos **sin necesidad de fijar  $K$**  de antemano, y genera una estructura jerárquica fácil de visualizar. El método más común es el **aglomerativo** (de abajo a arriba):

- 1 Se empieza tratando cada observación individual como su propio *cluster*.
- 2 En cada paso, se calculan las distancias entre todos los *clusters* y se **fusionan los dos más cercanos**.
- 3 El proceso se repite hasta que todas las muestras quedan integradas en un único *cluster* global.

# Dendrogramas e interpretación de cortes

La jerarquía de fusiones se representa gráficamente mediante un **dendrograma**: un diagrama de árbol donde el eje vertical indica la distancia (o disimilitud) a la que se fusionó cada par de grupos.

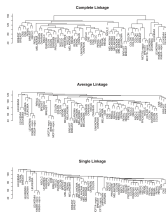
El usuario puede elegir el número final de *clusters* haciendo un **corte horizontal** a una altura concreta: el número de líneas que atraviesa el corte es el número de grupos resultantes. Esto ofrece una flexibilidad que algoritmos rígidos como *K-means* no tienen.



# Criterios de enlace (*linkage*)

Para decidir qué *clusters* fusionar en cada paso, hace falta definir cómo medir la distancia entre grupos (no entre puntos individuales):

- **Enlace completo:** la distancia **máxima** entre cualquier par de puntos de los dos grupos. Produce *clusters* compactos y bien balanceados.
- **Enlace simple:** la distancia **mínima** entre cualquier par. Es sensible al ruido y puede producir un efecto de “encadenamiento” (grupos alargados y difusos).
- **Enlace promedio:** el promedio de **todas** las distancias entre puntos de ambos grupos. Equilibra la estabilidad del enlace completo con la tolerancia del simple.



# DBSCAN: fundamentos y parámetros

El algoritmo **DBSCAN** (*Density-Based Spatial Clustering of Applications with Noise*) define los grupos según la **densidad local** de los datos, en vez de por cercanía a un centroide o por jerarquía. Esto le permite descubrir *clusters* de formas geométricas arbitrarias y aislar el ruido de forma natural.

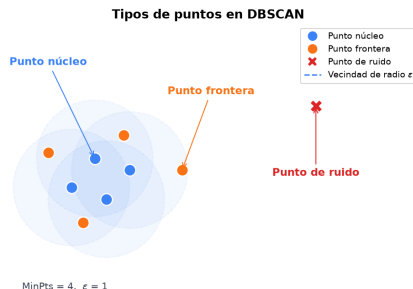
Requiere dos hiperparámetros:

- $\epsilon$  (**épsilon**): el radio de la vecindad alrededor de cualquier punto.
- **MinPts**: el número mínimo de puntos que debe haber dentro de ese radio para considerar la región “densa”.

# DBSCAN: clasificación de puntos

Durante su ejecución, DBSCAN clasifica cada punto en una de tres categorías según la densidad de su vecindad:

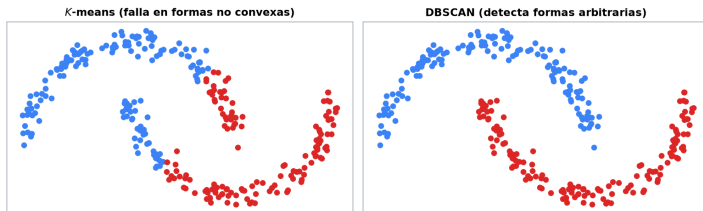
- **Puntos núcleo:** su vecindad de radio  $\epsilon$  contiene al menos  $\text{MinPts}$  muestras.
- **Puntos frontera:** no llegan a  $\text{MinPts}$  por sí mismos, pero están dentro de la vecindad de un punto núcleo.
- **Puntos de ruido:** no son ni núcleo ni frontera — se consideran anomalías y no se asignan a ningún *cluster*.



# DBSCAN: ventajas frente a $K$ -means

- **Robustez ante el ruido:**  $K$ -means obliga a cada muestra a pertenecer a un *cluster* (desplazando artificialmente los centroides ante valores atípicos); DBSCAN identifica y aísla el ruido de forma nativa.
- **Flexibilidad geométrica:** no asume que los *clusters* deban ser esféricos, por lo que puede descubrir formas alargadas, anidadas o irregulares.

$K$ -means frente a DBSCAN en datos con forma de luna

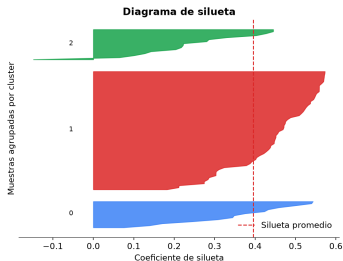


# Evaluación: el coeficiente de silueta

Sin etiquetas, ¿cómo sabemos si un *clustering* es bueno? El **coeficiente de silueta** compara, para cada punto, lo cerca que está de su propio grupo frente a lo cerca que está del grupo vecino más próximo:

$$sc(i) = \frac{b_i - a_i}{\max(a_i, b_i)}$$

El valor va de  $-1$  a  $1$ : cerca de  $+1$  indica un punto bien agrupado y alejado de otros *clusters*; cerca de  $0$ , un punto en la frontera entre dos grupos; y negativo sugiere que el punto está mal asignado.



# Evaluación: el estadístico *gap*

El **estadístico *gap*** es otra forma de elegir el número óptimo de *clusters*, más rigurosa que el método del codo: compara la inercia observada en los datos reales con la que se esperaría si los datos no tuvieran ninguna estructura de agrupamiento.

Su gran ventaja: puede detectar cuándo la respuesta correcta es que **no hay ningún agrupamiento real** en los datos — algo que el método del codo o la silueta no siempre dejan claro.

Existen más formas de evaluar un *clustering*, sobre todo cuando se usan modelos probabilísticos:

- **AIC y BIC**: comparan modelos penalizando los que tienen demasiados parámetros, para no quedarnos siempre con el más complejo.
- **Correlación cofenética**: comprueba si un dendrograma refleja bien las distancias reales entre los datos.

En la práctica, estos criterios rara vez se calculan a mano: las librerías de *machine learning* los aplican automáticamente para comparar modelos.

Cuando sí disponemos de etiquetas de referencia (aunque no se hayan usado para entrenar), podemos comprobar qué tan bien encajan los grupos descubiertos con las categorías reales:

- **Pureza:** a cada *cluster* se le asigna la etiqueta más frecuente en su interior, y se mide la proporción de aciertos así obtenida. Su punto débil: no penaliza tener demasiados *clusters* pequeños.
- **Índice Rand ajustado (ARI):** compara si el modelo agrupó las muestras de forma parecida a como lo harían las clases reales, corrigiendo el resultado que se obtendría por puro azar.

1 Introducción

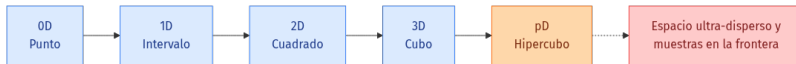
2 3.1 Clustering

3 3.2 Reducción de dimensionalidad

4 3.3 Detección de anomalías

# La maldición de la dimensionalidad

En la analítica moderna es habitual encontrar conjuntos de datos con cientos o miles de variables. Trabajar directamente en esos espacios de alta dimensión tiene graves inconvenientes, conocidos como la **maldición de la dimensionalidad**: a medida que aumenta el número de dimensiones, el volumen del espacio crece exponencialmente, y cualquier conjunto de datos real —por grande que sea— lo puebla de forma extremadamente **dispersa**. Las muestras dejan de tener “vecinos” cercanos de verdad, y las distancias entre puntos se vuelven casi indistinguibles.

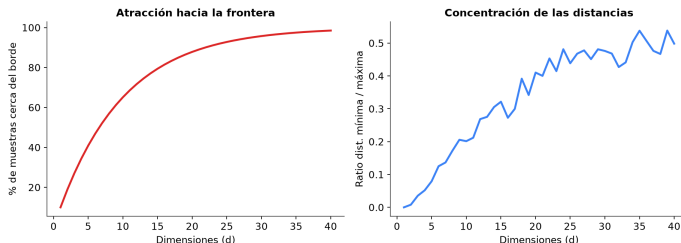


# ¿Por qué reducir dimensiones?

La reducción de dimensionalidad aborda este problema proyectando o comprimiendo los datos en un espacio de baja dimensión (habitualmente 2D o 3D). Esto ofrece varias ventajas:

- Acelera drásticamente la computación posterior.
- Mitiga el riesgo de *overfitting*.
- Es indispensable para **visualizar** datos complejos y para extraer factores latentes explicables.

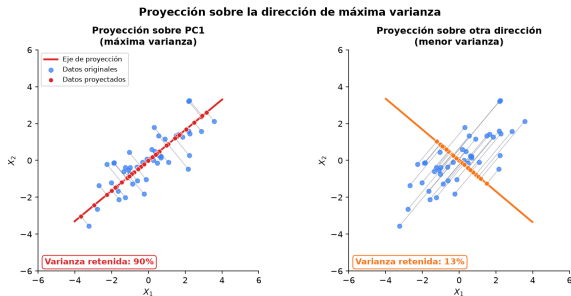
La maldición de la dimensionalidad, cuantificada



# PCA: la dirección de máxima varianza

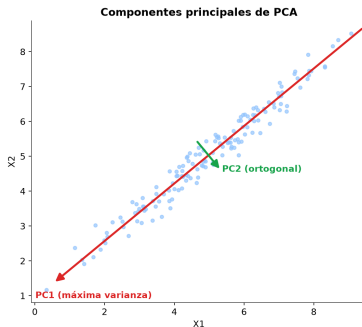
El **Análisis de Componentes Principales** (PCA) es el algoritmo de reducción de dimensionalidad lineal por excelencia. Proyecta ortogonalmente los datos originales sobre un subespacio de menor dimensión, minimizando la pérdida de información.

La **primera componente principal** (PC1) es la dirección del espacio de características a lo largo de la cual los datos **varían más**. Al proyectar los datos sobre este eje, se conserva el mayor porcentaje posible de la información original. La **segunda componente** (PC2) busca la dirección que explica la mayor varianza *restante*, con la condición de ser ortogonal a la primera.



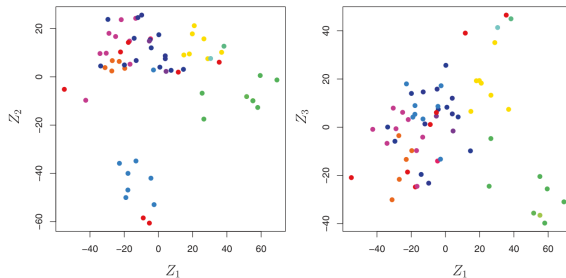
# PCA: direcciones principales y reconstrucción

PCA puede entenderse como un algoritmo de compresión: un “codificador” reduce cada muestra a una representación más pequeña (sus componentes principales), y un “decodificador” la reconstruye de vuelta al espacio original con el mínimo error posible. Cuantas menos componentes se conserven, mayor es la compresión, pero también mayor la información que se pierde.



# PCA en datos reales: NCI60 (6830 genes)

Un ejemplo real de la utilidad de PCA aparece en el conjunto de datos *NCI60*, donde cada muestra tiene 6830 genes (dimensiones). Al proyectar estas muestras sobre sus tres primeras componentes principales, las líneas celulares del mismo tipo de cáncer (mismo color) tienden a agruparse, pese a la enorme dimensionalidad original.



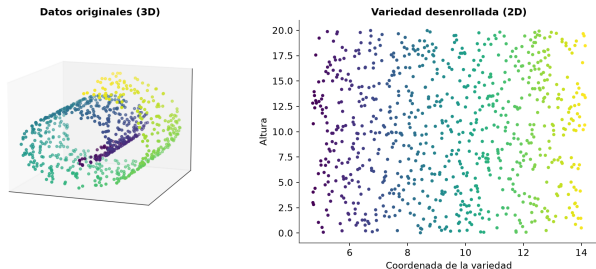
*Fuente: James, Witten, Hastie & Tibshirani (2013), An Introduction to Statistical Learning, Fig. 10.15.*

# Manifold learning: cuando los datos no son planos

PCA es rápido y robusto, pero asume que la estructura interesante de los datos es **lineal**. Cuando las relaciones son intrínsecamente no lineales (como en el clásico ejemplo del *Swiss roll*, un rollo de papel curvado en 3D), la proyección lineal de PCA colapsa puntos distantes entre sí.

La **hipótesis de la variedad** (*manifold hypothesis*) asume que los datos reales (imágenes, voces, textos) viven en una superficie curva de baja dimensión (una **variedad**), embebida dentro del espacio original de alta dimensión.

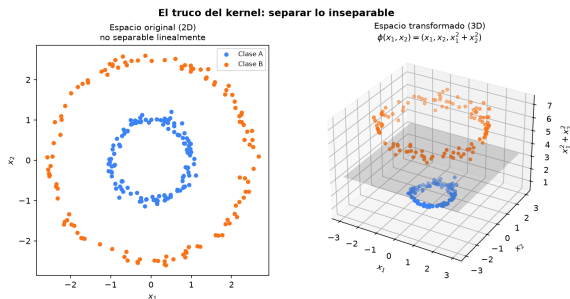
Swiss roll: motivación del manifold learning



## Kernel PCA: el truco del *kernel*

Para extender PCA a problemas no lineales, **kernel PCA** mapea implícitamente los datos de entrada a un espacio de dimensiones mucho más altas, donde relaciones que eran no lineales en el espacio original se vuelven **linealmente separables**.

El siguiente ejemplo clásico lo ilustra: dos clases en círculos concéntricos (izquierda) son imposibles de separar con una línea recta en 2D; tras aplicar el mapeo, quedan a alturas distintas y un simple plano las separa perfectamente.

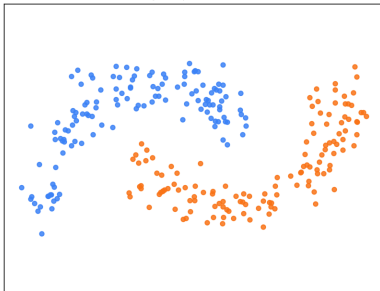


# $t$ -SNE: preservar vecindades locales

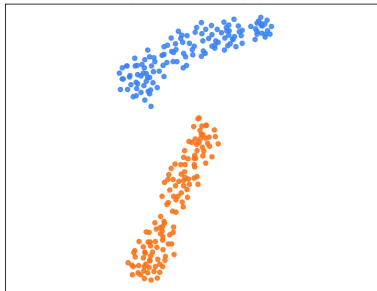
Propuesto por Maaten y Hinton (2008),  $t$ -SNE convierte las distancias entre muestras en **probabilidades de vecindad**: los puntos cercanos deben seguir cerca tras la proyección a 2D, y los lejanos, lejos. Es la técnica preferida para la **visualización cualitativa** de agrupamientos complejos, aunque resulta lenta en conjuntos de datos muy grandes.

PCA vs. t-SNE: dos lunas entrelazadas embebidas en 30D

PCA (proyección lineal)  
no logra separar las clases

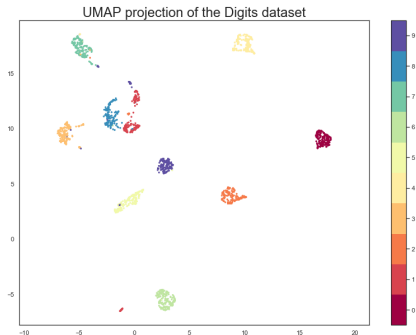


t-SNE (proyección no lineal)  
separa cada luna en un grupo



# UMAP: rápido y preciso

UMAP es una de las técnicas de *manifold learning* más potentes de la actualidad. A diferencia de *t-SNE*, que se centra casi exclusivamente en preservar vecindades muy locales, UMAP es capaz de preservar tanto la estructura **local** como la **global** de los datos, y además se ejecuta mucho más rápido sobre conjuntos de datos masivos, con millones de muestras.



*Fuente: umap-learn docs, Copyright (c) 2017 Leland McInnes, licencia BSD-3.*

1 Introducción

2 3.1 Clustering

3 3.2 Reducción de dimensionalidad

4 3.3 Detección de anomalías

# Filosofía de la detección de anomalías

La detección de anomalías es una tarea no supervisada de gran relevancia industrial: identificar patrones inusuales o eventos sospechosos dentro de un flujo de datos. Se aplica en la detección de fraudes financieros, la prevención de intrusiones en redes, el control de calidad en fabricación y la limpieza automática de conjuntos de datos.

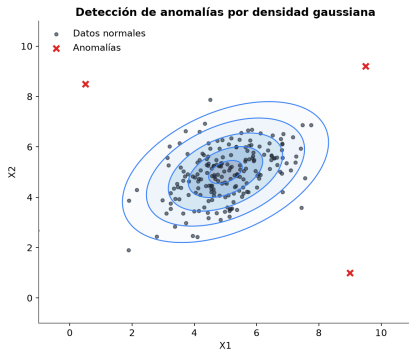
A nivel conceptual, se basa en que los comportamientos normales (*inliers*) son muy frecuentes y consistentes, mientras que las anomalías (*outliers*) son eventos raros que se desvían significativamente de la norma. Se distinguen dos escenarios según la “limpieza” de los datos de entrenamiento:

- **Anomaly detection:** el conjunto de entrenamiento puede contener ruido u *outliers* sin depurar.
- **Novelty detection:** el conjunto de entrenamiento está completamente limpio; el objetivo es detectar patrones nuevos nunca vistos.

**Ejemplo:** fraude bancario, fallos en una línea de producción, o un sensor que empieza a dar lecturas extrañas: en ninguno de estos casos sabemos de antemano cómo será la próxima anomalía, por eso se aborda como un problema no supervisado.

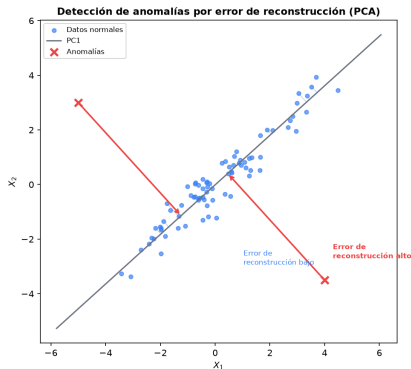
# Enfoque gaussiano

Se asume que las observaciones normales se concentran en regiones del espacio de características con **alta densidad de probabilidad**. Se ajusta una distribución gaussiana a los datos normales, y cualquier punto con una densidad de probabilidad por debajo de un umbral se marca como anomalía. El umbral puede fijarse, por ejemplo, a partir de la tasa histórica conocida de casos raros (un 4% de productos defectuosos, digamos).



# Enfoque de reconstrucción con PCA

Esta técnica se basa en que los componentes principales mayoritarios de PCA capturan las direcciones que caracterizan el comportamiento **normal** del sistema. Se proyectan los datos a baja dimensión con PCA y se reconstruyen de vuelta al espacio original: como las componentes principales no capturan las desviaciones inusuales, las anomalías presentarán un **error de reconstrucción** mucho mayor que las instancias normales.

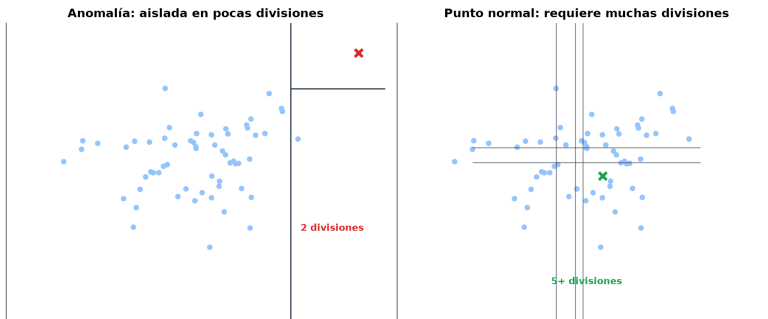


# Isolation Forest

A diferencia de los métodos anteriores, que modelan la densidad de los datos normales, **Isolation Forest** busca **aislar** directamente cada observación mediante particiones aleatorias: en cada nodo de un árbol se elige al azar una característica y un umbral de corte, dividiendo los datos en dos.

La intuición es simple: las anomalías, al estar alejadas del resto, requieren **muy pocas particiones** para quedar aisladas en su propia hoja, mientras que un punto normal necesita muchas más divisiones.

Isolation Forest: intuición del aislamiento

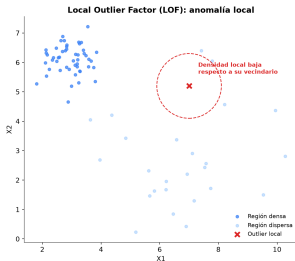


# Local Outlier Factor (LOF)

LOF compara la **densidad local** de una muestra con la de sus vecinos más cercanos. Una instancia normal tiene una densidad similar a su entorno; un *outlier local* presenta una densidad mucho menor que sus vecinos inmediatos.

Este método es muy útil para detectar **anomalías locales** que no destacarían en un análisis global, porque sus valores absolutos no son extremos, pero sí resultan inusuales dentro de su vecindario específico.

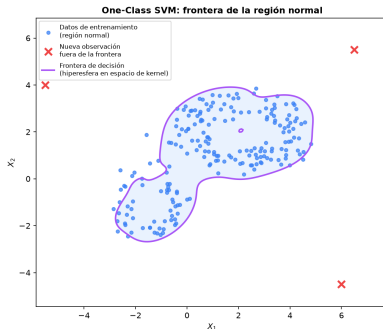
**Ejemplo:** una casa de 200 m<sup>2</sup> no es rara a nivel nacional, pero sí lo es en un barrio donde todas las casas miden 60 m<sup>2</sup>: eso es lo que detecta LOF.



# One-class SVM

Este algoritmo está optimizado para *novelty detection*, cuando se dispone de un conjunto de datos limpio para entrenar. En lugar de buscar un hiperplano que separe dos clases, proyecta los datos a un espacio de alta dimensión mediante un *kernel* y busca la región de **volumen mínimo** que encierra casi todas las muestras de entrenamiento.

Cualquier observación nueva que caiga fuera de esa región delimitada se clasifica automáticamente como anomalía o novedad.



- El aprendizaje no supervisado trabaja **sin etiquetas**, descubriendo la estructura latente de los propios datos.
- ***K-means*** agrupa por centroides (rápido, requiere  $K$  conocido); el **jerárquico** no necesita  $K$  y ofrece un dendrograma visual; **DBSCAN** detecta formas arbitrarias y aísla el ruido.
- El **coeficiente de silueta** y el **estadístico *gap*** son las herramientas más habituales para evaluar un *clustering* sin etiquetas.
- La **maldición de la dimensionalidad** motiva reducir el número de variables antes de analizar o visualizar datos.
- **PCA** es rápido y lineal; **kernel PCA**, ***t-SNE*** y **UMAP** capturan estructuras no lineales, con distintos equilibrios entre velocidad y fidelidad.
- La detección de **anomalías** combina enfoques estadísticos (gaussiano, reconstrucción con PCA) y algoritmos específicos (*Isolation Forest*, LOF, *one-class SVM*).

Este material se distribuye bajo licencia

## Creative Commons Reconocimiento-NoComercial 4.0

(CC BY-NC 4.0)

### Eres libre de:

- **Compartir:** copiar y redistribuir el material en cualquier medio o formato.
- **Adaptar:** remezclar, transformar y crear a partir del material.

Por ejemplo: usarlas en clase, subirlas al aula virtual, traducirlas o reutilizar una figura en tus apuntes.

### Con estas condiciones:

- **Reconocimiento (BY):** debes citar la autoría, enlazar a la licencia e indicar si has hecho cambios.
- **NoComercial (NC):** no puedes usar el material con fines comerciales.

No puedes añadir restricciones que impidan a otros hacer lo que la licencia permite.

[creativecommons.org/licenses/by-nc/4.0/deed.es](https://creativecommons.org/licenses/by-nc/4.0/deed.es)

# Capítulo 3: Aprendizaje No Supervisado

Universidad de Salamanca

Facultad de Ciencias

Programación Avanzada

Diego M. Jiménez-Bravo · [dmjimenez@usal.es](mailto:dmjimenez@usal.es)